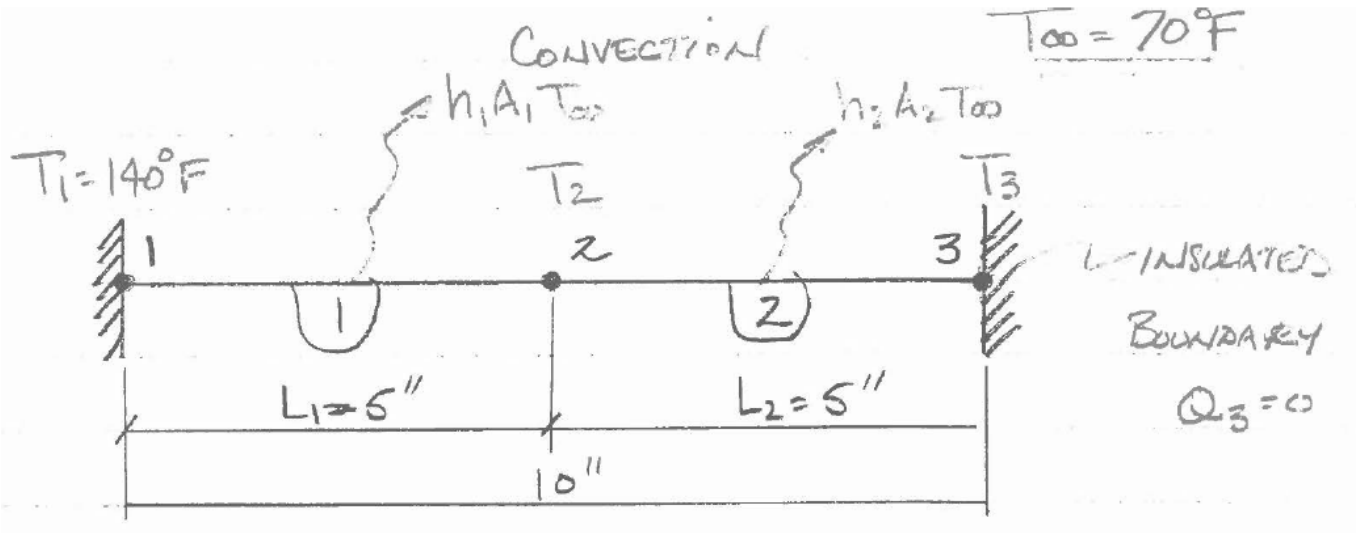
me5634 hw4: 1D Thermal Stress - Fixed-Fixed Bar

Table of Contents

Initialize the MATLAB workspace.....2
Define Problem Specific Element and Node Parameters.....2
Compute Problem Specific Parameters and Allocate Space.....2
Setup Element Result Table.....3
Define Element Nodal Coordinates and Assembly Vector.....3
Define Model Section Properties; Cross-Section Area.....3
Define Material Properties; Conduction, Convection Coeffs and Tinfinity.....3
Define Ambient and STress -Free Reference Temperatures.....4
Assemble Elements into Conduction and Convection Global Matrices.....4
Define Element Precsribed Temperature.....4
Solve System of Partitioned Equations for Static Equilibrium.....4
Post-Processing Phase of the Solution using Strain-Displ, Stress-Strain Approach.....5
Plot the Thermal Solution.....5
Compute Problem Specific Parameters and Allocate Space.....7
Allocate Global Array Storage.....7
Redefine the Assembly Vectors for the Structural Problem.....7
Assemble Element Stiiffness Matrix and Equivalent Joint Loads into Global Matrices.....7
Apply the Prescribed Displacement BCs.....8
Solve the Global System Equations for the Linear Static Analysis.....8
Postprocess the Solution.....8
Post Strains and Stresses.....9
Plot the Solution.....9

A fixed-fixed steel bar 10" long has a fixed temperture of 140 degF at the left boundary. The right boundary of the bar is completely insulated. The steel bar has a constant diamter of 3/4".



The steel bar has the following material properties:

Material Properties Steel:

- Elastic Modulus: $E = 30 \times 10^6$ psi
- Coefficient of Thermal Expansion: $\alpha = 6.0 \times 10^{-6}$ in/in-°F
- Thermal conductivity: $k_x = 1.6667$ BTU/(hr-in-°F)
- Stress-Free Reference Temperature: $T_{Reference} = T_{Ref} = 75$ °F

The bar operates in an ambient environment:

- Convection coefficient: $h = 0.0069$ BTU/(hr-in^2-°F)
- Ambient Temperature: $T_\infty = 70$ °F

Determine the thermal solution:

1. temperature distribution,
2. heat fluxes due to conduction and convection.

Determine the structural solution:

1. compute the axial displacement field,
2. the reactions at the fixed ends,
3. the total axial strain,
4. the thermal axial strains,
5. the elastic axial strains,
6. the axial stresses

Initialize the MATLAB workspace

```
clc;                                                % clear MATLAB command line
clear all;                                          % initialize MATLAB workspace
format short e;                                     % set format of output to
exponential
```

Define Problem Specific Element and Node Parameters

```
nElements = 2;                                      % number of elements
nNodes = nElements + 1;                             % number of nodes
ndofPerNode = 1;                                    % number of dof per node
nNodesPerElement = 2;                               % number of nodes per element
ndofPerElement = ndofPerNode * nNodesPerElement;    % number of dof per element
```

Compute Problem Specific Parameters and Allocate Space

```
nNodesFixedTemperature = 1;                         % number of nodes with fixed
temperature
ndofPrescribed = ndofPerNode*nNodesFixedTemperature;% number of prescribed dof
ndofTotal = nNodes * ndofPerNode;                   % number of total dof
```

```
ndofUnknown = ndofTotal - ndofPrescribed;           % number of unknown dof
```

Setup Element Result Table

```
nResults = 3;                                       % number of fields in element
output table
ResultTable = zeros(nElements,nResults);            % allocate space for element
table
```

Define Element Nodal Coordinates and Assembly Vector

```
Ltotal = 10.0;                                      % bar total length [in]
xv = linspace(0.0, Ltotal, nNodes);                 % generate the nodal coordinate
vector [cm]
Xe = zeros(ndofPerElement, nElements);              % initialize element nodal
coordinates
Le = zeros(1,nElements);                            % initialize element lengths
assyv = zeros(ndofPerElement, nElements);           % initialize element assembly
vector
for Ei = 1:nElements
    Xe(:,Ei) = [xv(Ei); xv(Ei+1)];                  % Ei nodal coordinates
    Le(Ei)   = Xe(2,Ei) - Xe(1,Ei);                 % Ei length
    assyv(:,Ei) = [Ei-1; Ei];                       % Ei assembly vector
end
assyv(1,1) = ndofTotal;                             % reset prescribed temp
assembly index
```

Define Model Section Properties; Cross-Section Area

```
d = 0.75;                                           % diameter [in]
P = pi*d;                                           % nominal perimeter [in]
A = (pi/4)*d^2;                                     % nominal cross-section area
[in^2]
```

Define Material Properties; Conduction, Convection Coeffs and Tinfinity

```
% conduction
k = 1.6667;                                         % conduction coeff of copper
[(BTU/hr)/(in-degF)]
kA = k*A;                                           % conduction modulues [(BTU/hr)-
in/(degF)]
% convection
h = 0.0069;                                         % convection coeff of copper
[(BTU/hr)/(in^2*degF)]
hP = h*P;                                           % convection modulus [(BTU/hr)/
(hr-degF)]
% elasticity
E = 30.0e6;                                         % elastic mudulus [psi]
AE = A*E;                                           % axial modulus [lbf]
% thermal strain
```

```
alpha = 6.0e-6;                                    % coeff of thermal expansion
[in/(in-degF)]
```

Define Ambient and STress -Free Reference Temperatures

```
Tinf = 70.0;                                       % ambient temperature [degF]
Tref = 70.0;                                       % stress-free reference
temperature [degF]
```

Assemble Elements into Conduction and Convection Global Matrices

```
% global matrices and vectors
Kg = zeros(ndofTotal,ndofTotal);                   % initialize global system
matrix
Tg = NaN*ones(ndofTotal,1);                        % initialize global temperature
vector
Qg = zeros(ndofTotal,1);                           % initialize global flux vector
% elemental matrices and vectors
keCond = zeros(ndofPerElement,ndofPerElement,nElements);                %
initialize element conduction matrices
keConv = zeros(ndofPerElement,ndofPerElement,nElements);                %
initialize element convection matrices
ke     = zeros(ndofPerElement,ndofPerElement,nElements);                %
initialize element conduction/convection matrices
QeConv = zeros(ndofPerElement, nElements);                              %
initialize element convection load
% do for each element
for Ei = 1:nElements
    keCond(:,:,Ei) = heat1D_AxialConductionMatrix(kA, Le(Ei));          % Ei axial
conduction matrix
    keConv(:,:,Ei) = heat1D_AxialConvectionMatrix(hP, Le(Ei));          % Ei axial
convection matrix
    ke(:,:,Ei) = keCond(:,:,Ei) + keConv(:,:,Ei);                       % Ei
conduction/convection matrix
    Kg = directStiffness_AssembleMatrix(Kg, ke(:,:,Ei), assyv(:,Ei));   % assemble
E1 stiffness
    QeConv(:,Ei) = heat1D_ConvectionLoadVector(hP, Le(Ei), Tinf);       % Ei
convection load
    Qg = directStiffness_AssembleVector(Qg, QeConv(:,Ei), assyv(:,Ei)); % assemble
E1 nodal heat fluxes
end
```

Define Element Precsribed Temperature

```
Tg(ndofTotal,1) = 140.0;                           % prescribe T1 = 140.0 degF
```

Solve System of Partitioned Equations for Static Equilibrium

```
[Tu, Qr, Tg, Qg] = solver_Static(ndofPrescribed, ndofTotal, Kg, Tg, Qg)   % solve
equilibrium eqns
```

```
Tu = 2×1
   1.0802e+02
   9.9159e+01
Qr =
   7.1211e+00
Tg = 3×1
   1.0802e+02
   9.9159e+01
   1.4000e+02
Qg = 3×1
   5.6902e+00
   2.8451e+00
   7.1211e+00
```

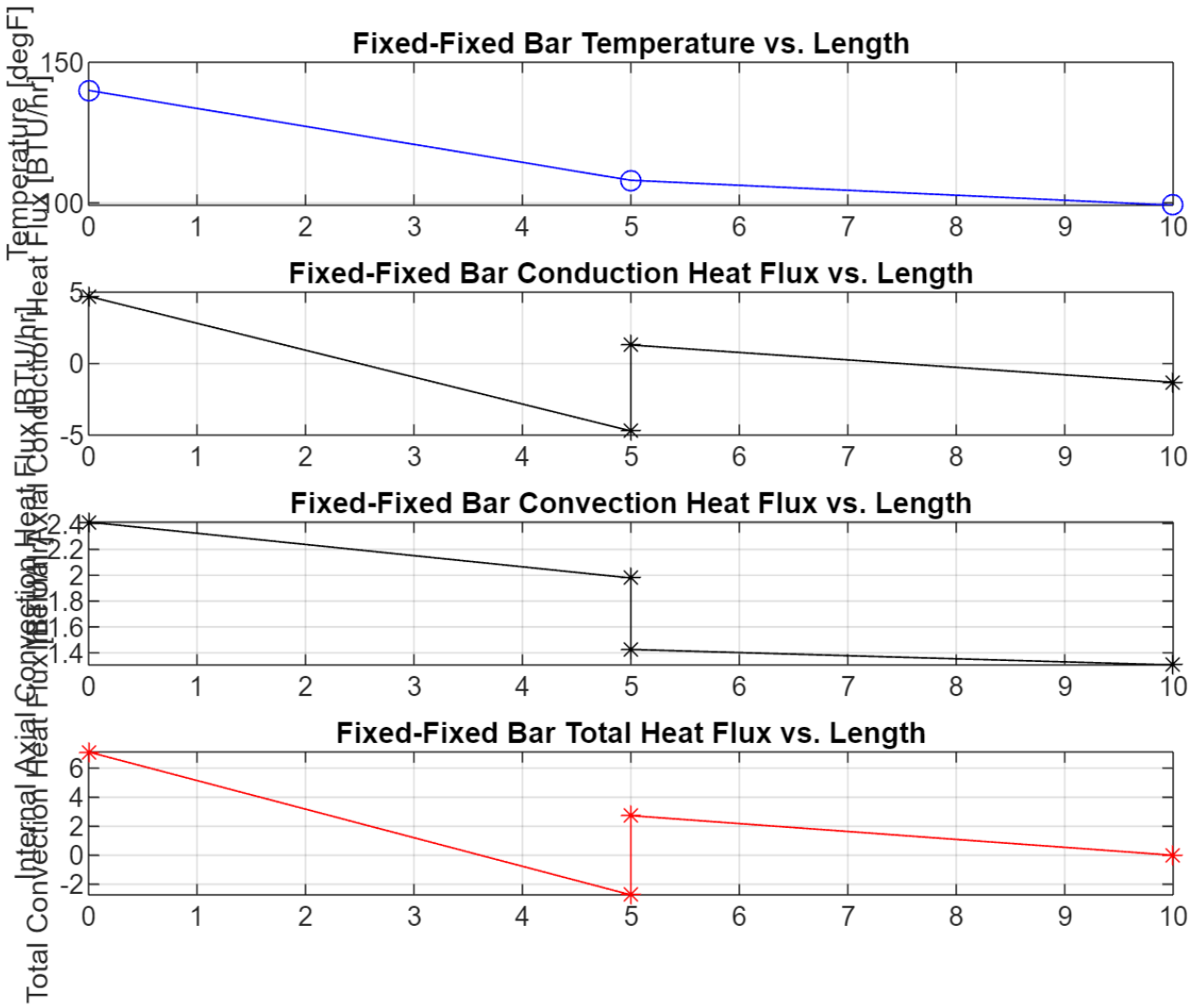
Post-Processing Phase of the Solution using Strain-Displ, Stress-Strain Approach

```
TinfV = Tinf*ones(ndofPerElement,1);             % T infinity vector
np = 0;                                             % initialize number of plot
points
Energy = zeros(nNodes,1);                           % zero out nodal energy balance
%   do for each element
for Ei = 1:nElements
    Te(:,Ei) = directStiffness_ExtractElementVectorFromAssembledVector(Tg,
assyv(:,Ei)); % Ei global displacements
    QConde(:,Ei) = keCond(:,:,Ei)*Te(:,Ei);          % element internal conduction
heat flux
    QConve(:,Ei) = keConv(:,:,Ei)*(Te(:,Ei) - TinfV);% element internal convection
heat flux
    QTotale(:,Ei) = QConde(:,Ei) + QConve(:,Ei);     % element internal total heat
flux
    %  Post-Processing for Plotting
    %  do for each node in the element
    for Ni = 1:nNodesPerElement
        np = np + 1;                                          % Ei Ni index
        xPlot(np) = Xe(Ni,Ei);                                % Ei Ni coordinate
        tPlot(np) = Te(Ni,Ei);                                % Ei Ni temperature
        qCondPlot(np) = QConde(Ni,Ei);                        % Ei Ni internal axial
conduction heat flux
        qConvPlot(np) = QConve(Ni,Ei);                        % Ei Ni internal axial
convection heat flux
        qTotalPlot(np) = QTotale(Ni,Ei);                      % Ei Ni internal axial
total heat flux
    end
    Energy = directStiffness_AssembleVector(Energy, QTotale(:,Ei), assyv(:,Ei));
% assemble E1 nodal heat fluxes;   % Ei N1
end
```

Plot the Thermal Solution

```
figure(2);                                                  % plot as figure 2
subplot (4,1,1);                                            % axial
displacement subplot
```

```
plot(xPlot, tPlot,'b-o'), grid;                                    % plot temperature
title('Fixed-Fixed Bar Temperature vs. Length');                   % plot title
ylabel('Temperature [degF]');                                      % y-axis label
subplot(4,1,2);                                                    % internal axial
force subplot
plot(xPlot, qCondPlot, 'k-*'), grid;                               % plot axial
internal heat flux due to conduction
title('Fixed-Fixed Bar Conduction Heat Flux vs. Length'); % plot title
ylabel('Internal Axial Conduction Heat Flux [BTU/hr]');            % y-axis label
subplot(4,1,3);                                                    % internal axial
force subplot
plot(xPlot, qConvPlot, 'k-*'), grid;                               % plot axial
internal heat flux due to convection
title('Fixed-Fixed Bar Convection Heat Flux vs. Length');          % plot title
ylabel('Internal Axial Convection Heat Flux [BTU/hr]');            % y-axis label
subplot(4,1,4);                                                    % internal axial
force subplot
plot(xPlot, qTotalPlot, 'r-*'), grid;                              % plot axial
internal heat flux due to convection
title('Fixed-Fixed Bar Total Heat Flux vs. Length');               % plot title
ylabel('Internal Total Convection Heat Flux [BTU/hr]');            % y-axis label
```



Compute Problem Specific Parameters and Allocate Space

```
nNodesFixed = 2;                                % number of nodes fixed
ndofPrescribed = ndofPerNode*nNodesFixed;       % number of prescribed dof
ndofTotal = nNodes * ndofPerNode;               % number of total dof
ndofUnknown = ndofTotal - ndofPrescribed;       % number of unknown dof
```

Allocate Global Array Storage

```
Kg = zeros(ndofTotal,ndofTotal);                % allocate space for ndof x
ndof assembled stiffness
Ug = NaN*ones(ndofTotal,1);                     % allocate space for ndof x 1
assembled displ vector
Fg = zeros(ndofTotal,1);                        % allocate space for ndof x 1
assembled load vector
```

Redefine the Assembly Vectors for the Structural Problem

```
assyv = zeros(ndofPerElement, nElements);       % allocate arrays for element
assembly vector
for Ei = 1:nElements
    assyv(1,Ei) = Ei-1;                         % N1 internal assembly index
    assyv(2,Ei) = Ei;                           % N2 internal assemb;y index
end
%   Assemble BC dof
assyv(1,1) = ndofTotal-1;                       % define E1, N1 assembly vector
assyv(2,nElements) = ndofTotal;                 % define En, N2 assembly vector
```

Assemble Element Stiiffness Matrix and Equivalent Joint Loads into Global Matrices

```
ke = zeros(ndofPerElement, ndofPerElement, nElements);  % initialize element local
stiffness matrix
ue = NaN*ones(nNodesPerElement, nElements);             % initialize element
prescribed displacements% do for each element
Fe = zeros(nNodesPerElement, nElements);                % initialize element
external nodal loading
Tavg = zeros(1, nElements);                             % initialize element
average temperatures
dTavg = zeros(1, nElements);                            % initialize element
avergage delta temperature
alpha_dT = zeros(1, nElements);                         % initialize element
alpha*dT
for Ei = 1:nElements
    Tavg(Ei) = mean(Te(:,Ei));                                  % Ei
average temperature
    dTavg(Ei) = Tavg(Ei) - Tref;                                % Ei delta T
    alpha_dT(Ei) = alpha * dTavg(Ei);                           % Ei
thermal strain distribution
```

```
    ke(:,:,Ei) = bar1D_AxialStaticStiffnessMatrix(AE, Le(Ei));          % compute
stiffness matrix for all
    Kg = directStiffness_AssembleMatrix(Kg, ke(:,:,Ei), assyv(:,Ei));   % Ei
assemble stiffness
    Ug = directStiffness_AssembleVector(Ug, ue, assyv(:,Ei));           % Ei
assemble nodal displacement
    Fe = bar1D_ThermalNodalLoadVector(AE, alpha_dT(Ei));                % Ei
applied thermal nodal loading
    Fg = directStiffness_AssembleVector(Fg, Fe, assyv(:,Ei));           % Ei
assemble nodal loading
end
```

Apply the Prescribed Displacement BCs

```
Ug(ndofTotal-1) = 0.0;                      % fixed left bc
Ug(ndofTotal) = 0.0;                        % fixed right bc
```

Solve the Global System Equations for the Linear Static Analysis

```
[Uu, Fr, Ug, Fg] = solver_Static(ndofPrescribed, ndofTotal, Kg, Ug, Fg);
```

Postprocess the Solution

using Strain-Displ, Stress-Strain Approach:

do for each element Ei

Extract the element displacements from the assembled displacements for Ei: $\{U_g\} \Rightarrow \{u^{Ei}\}$

Compute the Total Strain for Ei:

Compute the Thermal Strain for Ei: $\{\varepsilon^{Ei}\} = \alpha \Delta T_{avg} \begin{Bmatrix} +1 \\ +1 \end{Bmatrix}$

Compute the Elastic Strain for Ei: $\left\{\varepsilon^{Ei}_{Elastic}\right\} = \left\{\varepsilon^{Ei}_{Total}\right\} - \left\{\varepsilon^{Ei}_{Thermal}\right\}$

Compute the Elastic Stresses for Ei: $\left\{\sigma^{Ei}_{Elastic}\right\} = [E^{Ei}]\left\{\varepsilon^{Ei}_{Elastic}\right\}$

end

```
np = 0;                                                     % initialize number of
plot points
%   do for each element
for Ei = 1:nElements
    ue(:,Ei) = directStiffness_ExtractElementVectorFromAssembledVector(Ug,
assyv(:,Ei)); % Ei global displacements
    fe(:,Ei) = ke(:,:,Ei)*ue(:,Ei);                         % Ei interal axial
forces
    B = bar1D_AxialStrainDisplacementMatrix(Le(Ei));        % Ei strain-displacemnt
matrix
    StrainTotal(:,Ei)   = B*ue(:,Ei);                       % Ei total strain
    StrainThermal(:,Ei) = alpha_dT(Ei)*[+1; +1];            % Ei thermal strain
```

```
    StrainElastic(:,Ei) = StrainTotal(:,Ei) - StrainThermal(:,Ei);  % Ei elastic
strains
    StressElastic(:,Ei) = E*StrainElastic(:,Ei);            % Ei stress
    %  Post-Processing for Plotting
    %  do for each node in the element
    for Ni = 1:nNodesPerElement
        np = np + 1;                                                % Ei Ni index
        xPlot(np) = Xe(Ni,Ei);                                      % Ei Ni coordinate
        uPlot(np) = ue(Ni,Ei);                                      % Ei Ni axial
displacement
        fPlot(np) = fe(2,Ei);                                       % Ei Ni internal
axial force
        eTotalPlot(np) = StrainTotal(Ni,Ei);                        % Ei Ni axial strain
        eThermalPlot(np) = StrainThermal(Ni,Ei);                    % Ei Ni axial strain
        eElasticPlot(np) = StrainElastic(Ni,Ei);                    % Ei Ni axial strain
        sPlot(np) = StressElastic(Ni,Ei);                           % Ei node 1 axial
stress
    end
end
```

Post Strains and Stresses

```
fprintf('Fixed-Fixed Bar Results - Thermal Loading');
```

```
Fixed-Fixed Bar Results - Thermal Loading
```

```
fprintf(' Ei   Axial Force  Total Strain  Thermal Strain   Elastic Strain  Axial
Stress');  % table header
```

```
 Ei   Axial Force  Total Strain  Thermal Strain   Elastic Strain  Axial Stress
```

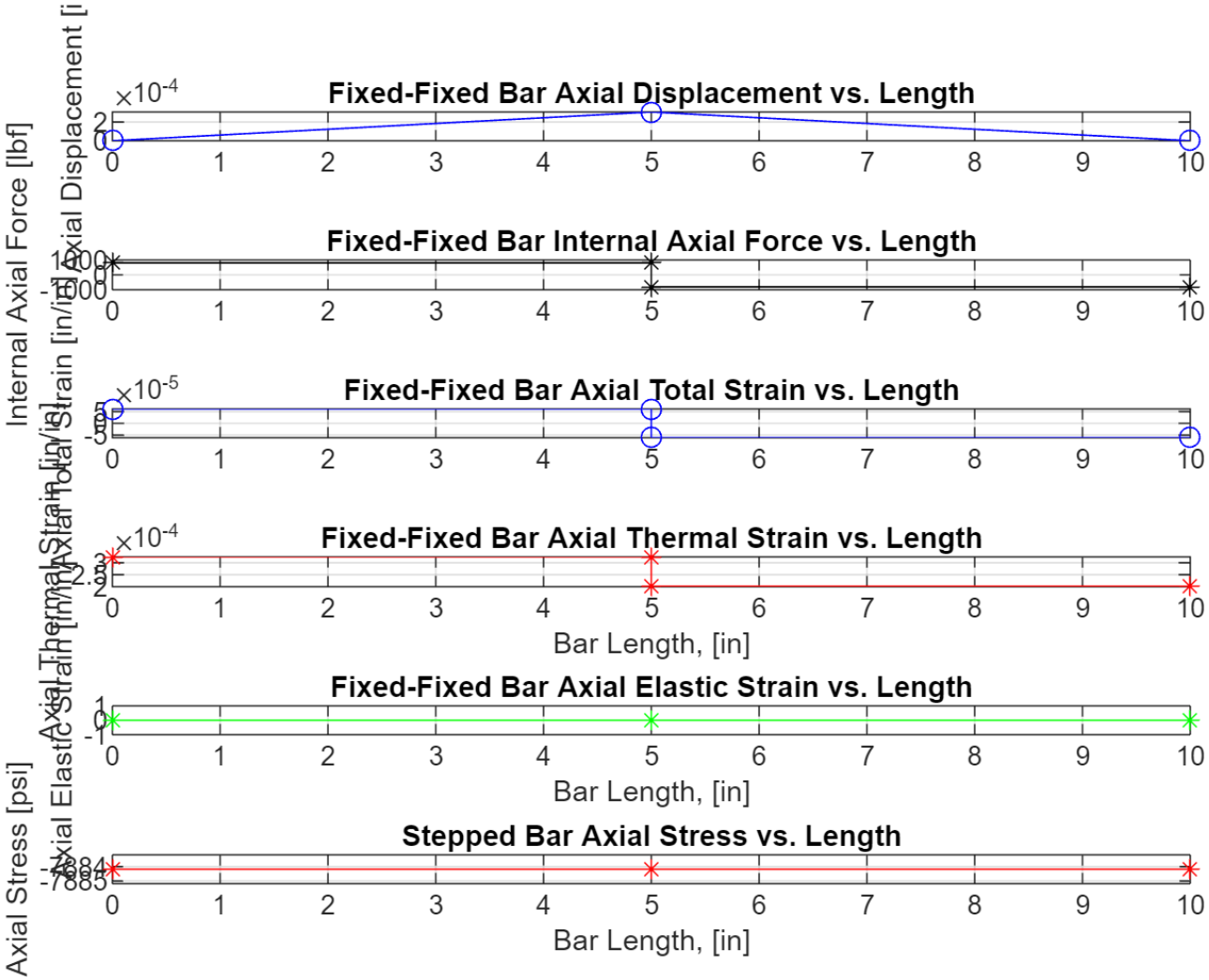
```
resultFormat = '%3d  %+12.2f  %+12.4e   %+12.4e    %+12.4e  %+12.2f\n';     %
result table format
for Ei = 1:nElements
    fprintf(resultFormat, Ei, fe(2,Ei), StrainTotal(2,Ei), StrainThermal(2,Ei),
StrainElastic(2,Ei), StressElastic(2,Ei));       % post Ei result
end
```

```
  1      +811.93   +6.1261e-05    +3.2407e-04     -2.6281e-04      -7884.18
  2      -811.93   -6.1261e-05    +2.0154e-04     -2.6281e-04      -7884.18
```

Plot the Solution

```
figure(3);                                                          % plot as figure 2
subplot (6,1,1);                                                    % axial
displacement subplot
plot(xPlot, uPlot,'b-o'), grid;                                     % plot axial
displacement
title('Fixed-Fixed Bar Axial Displacement vs. Length');             % plot title
ylabel('Axial Displacement [in]');                                  % y-axis label
subplot(6,1,2);                                                     % internal axial
force subplot
```

```
plot(xPlot, fPlot, 'k-*'), grid;                                % plot axial
internal force
title('Fixed-Fixed Bar Internal Axial Force vs. Length');       % plot title
ylabel('Internal Axial Force [lbf]');                           % y-axis label
subplot(6,1,3);                                                 % axial strain
subplot
plot(xPlot, eTotalPlot, 'b-o'), grid;                           % plot axial total
strain
title('Fixed-Fixed Bar Axial Total Strain vs. Length');         % plot title
ylabel('Axial Total Strain [in/in]');                           % y-axis label
subplot(6,1,4);                                                 % axial stress
subplot
plot(xPlot, eThermalPlot, 'r-*'), grid;                         % plot axial stress
ylabel('Axial Thermal Strain [in/in]');                         % y-axis label
title('Fixed-Fixed Bar Axial Thermal Strain vs. Length');       % plot title
xlabel('Bar Length, [in]');                                     % x-axis label
subplot(6,1,5);                                                 % axial stress
subplot
plot(xPlot, eElasticPlot, 'g-*'), grid;                         % plot axial
elastic strain
ylabel('Axial Elastic Strain [in/in]');                         % y-axis label
title('Fixed-Fixed Bar Axial Elastic Strain vs. Length');       % plot title
xlabel('Bar Length, [in]');                                     % x-axis label
subplot(6,1,6);                                                 % axial stress
subplot
plot(xPlot, sPlot, 'r-*'), grid;                                % plot axial stress
ylabel('Axial Stress [psi]');                                   % y-axis label
title('Stepped Bar Axial Stress vs. Length');                   % plot title
xlabel('Bar Length, [in]');                                     % x-axis label
```


Fixed-Fixed Bar Axial Displacement vs. Length
Axial Displacement [i
Fixed-Fixed Bar Internal Axial Force vs. Length
Internal Axial Force [lbf]
Fixed-Fixed Bar Axial Total Strain vs. Length
Axial Total Strain [in/in]
Fixed-Fixed Bar Axial Thermal Strain vs. Length
Axial Thermal Strain [in/in]
Bar Length, [in]
Fixed-Fixed Bar Axial Elastic Strain vs. Length
Axial Elastic Strain [in/in]
Bar Length, [in]
Stepped Bar Axial Stress vs. Length
Axial Stress [psi]
Bar Length, [in]